Project Report

on

AI-Powered Flower Classification and Dominant Color Detection

(Utilizing MobileNetV2 and K-Means Clustering for Real-Time Botanical Analysis)

Submitted by : Ritika Rajak

Roll No : 23051126

Course : Computer Science Engineering

Date : March 2026

Abstract

This project presents a dual-pipeline machine learning web application for real-time botanical analysis. The application uses Transfer Learning with the MobileNetV2 Convolutional Neural Network (CNN) for lightweight, high-accuracy species classification. Concurrently, a computer vision pipeline applies unsupervised K-Means clustering to calculate its exact dominant RGB color. Built with Streamlit, the system successfully delivers simultaneous species identification and real-world color detection, demonstrating the power of combining deep neural networks with unsupervised machine learning.

Introduction

1. Project Overview This project presents a machine learning web application capable of simultaneously identifying the species of a flower and extracting its dominant real-world color. Recognizing flowers and their phenotypic traits (like color) is highly valuable for botany, agriculture, and educational tools.

2. Problem Statement Standard image classification models can identify an object but fail to extract specific visual traits like true color, which is often distorted by background noise (leaves, dirt, shadows). This project solves that by splitting the task into two distinct pipelines: a deep learning classifier for the species, and an isolated computer vision algorithm for the color.

3. Technologies Used

- **Deep Learning:** TensorFlow & Keras (MobileNetV2 architecture)
- **Computer Vision & Math:** OpenCV, Scikit-Learn (K-Means Clustering)
- **Frontend:** Streamlit (Python-based web framework)

Model Architecture

1. Dual-Pipeline System Overview Standard end-to-end classification models are highly effective at identifying objects, but they struggle to isolate and extract specific continuous phenotypic traits, such as precise petal color, due to interference from background noise. To solve this, the proposed architecture utilizes a modular, dual-pipeline approach. It separates the cognitive task of species identification from the computer vision task of color extraction, running a Deep Learning model and an Unsupervised Machine Learning algorithm in parallel.

2. Classification Architecture: MobileNetV2 To classify the flower species with high accuracy and low latency, the system employs Transfer Learning based on the **MobileNetV2** convolutional neural network architecture. The model was pre-trained on the ImageNet dataset, allowing it to leverage pre-learned feature extraction capabilities for basic shapes, edges, and textures.

- **Depthwise Separable Convolutions:** MobileNetV2 replaces computationally heavy 3D convolutions with a highly efficient two-step process. A depthwise convolution first applies independent spatial filters to each input channel, followed by a 1x1 pointwise convolution that mixes the channels to build complex features. This factorization drastically reduces parameter count, allowing the lightweight model to run seamlessly in real-time web environments without a dedicated GPU.
- **Custom Classification Head:** While the base feature extraction layers of MobileNetV2 were frozen to retain their foundational visual knowledge, the top layers were customized for this specific botanical dataset. A Global Average Pooling layer condenses the final feature maps, feeding into a fully connected `Dense` layer utilizing a Softmax activation function to output normalized probability scores for each flower class.

3. Unsupervised Color Detection Pipeline Simultaneously, the uploaded image passes through a sequential computer vision pipeline designed to extract the true dominant color of the flower petals while ignoring the stem, leaves, and surrounding environment.

- **K-Means Clustering:** The valid RGB pixels are fed into an unsupervised K-Means clustering algorithm. By explicitly setting the algorithm to find a single cluster center, the model evaluates all the isolated petal pixels and groups them together to find the most representative core color. This returns a single, highly accurate dominant RGB value.
- **Distance Mapping:** Finally, the extracted dominant RGB value is mapped against a custom dictionary of calibrated real-world colors. The algorithm calculates the shortest direct distance between the flower's RGB profile and each dictionary option, returning the human-readable color name that most closely matches the flower.

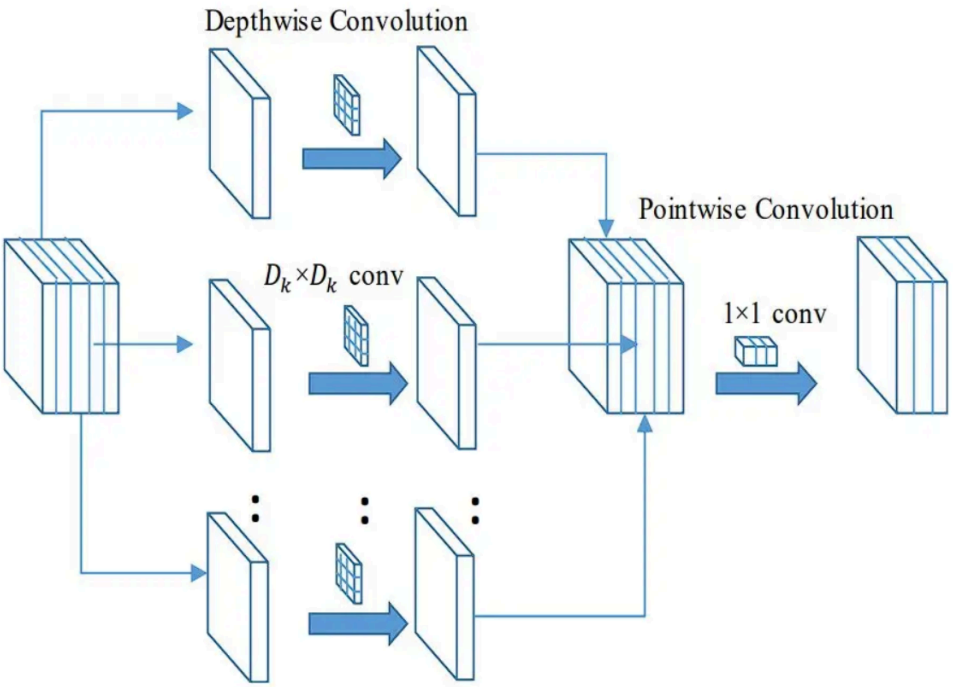

Figure 1: The architecture of a Depthwise Separable Convolution (used in MobileNetV2), demonstrating the two-step process of spatial filtering per channel followed by a 1x1 pointwise convolution for channel mixing.

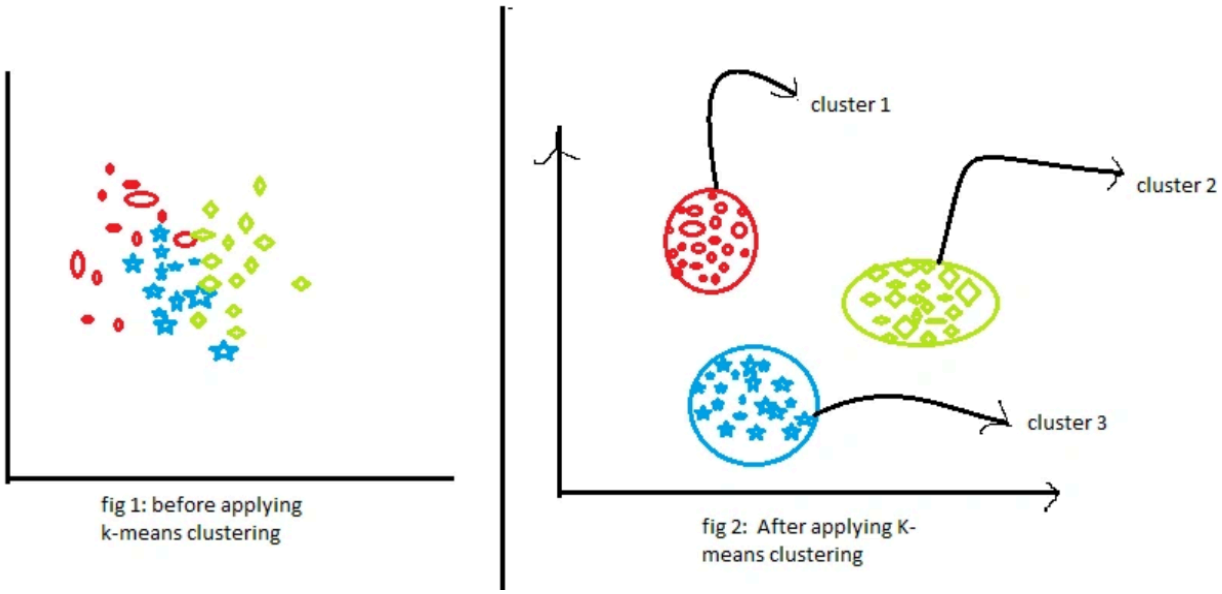

Figure 2: A visual representation of K-Means clustering. The algorithm processes unorganized data points (such as the RGB pixel values of a flower) and groups them into distinct, mathematically defined clusters to find the dominant center

Sample Code

1. Flower Classification (CNN)

```
st.subheader("1. Species Prediction")
# Resize the uploaded image to match the CNN input shape
img_resized = image.resize((180, 180))
# Convert image to array and add batch dimension for the model
img_array = tf.keras.preprocessing.image.img_to_array(img_resized)
img_array = tf.expand_dims(img_array, 0)
# Feed to the MobileNetV2 Transfer Learning model
predictions = model.predict(img_array)
score = tf.nn.softmax(predictions[0])
# Extract the predicted class and confidence percentage
predicted_class = class_names[np.argmax(score)]
confidence = 100 * np.max(score)
```

2. Color Detection (K-Means)

```
st.subheader("2. Color Detection")
    # 1. Create a mask to select only solid pixels (Alpha channel > 0)
    mask = img_no_bg_array[:, :, 3] > 0
    flower_pixels = img_no_bg_array[mask][:, :3]
if len(flower_pixels) > 0:
    # 2. Use K-Means Clustering to find the exact mathematical center
    kmeans = KMeans(n_clusters=1, n_init=10)
    kmeans.fit(flower_pixels)
    dominant_color = kmeans.cluster_centers_[0].astype(int)
    # 3. Find the closest human-readable name using Euclidean distance
    min_dist = float('inf')
    closest_name = "Unknown"
    for name, color_val in COLORS.items():
        dist = sum((a - b) ** 2 for a, b in zip(dominant_color, color_val))
        if dist < min_dist:
            min_dist = dist
            closest_name = name
    # 5. Display the color name and a visual color swatch
    st.write(f"**Dominant Color (RGB):** {dominant_color}")
    st.write(f"**Closest Match:** {closest_name}")
    st.markdown(
        f'<div style="width:50px;height:50px;'

f'background-color:rgb({dominant_color[0]},{dominant_color[1]},{dominant_col
or[2]}); '
        f'border: 1px solid black;"></div>',
        unsafe_allow_html=True
    )
```

Output

The application provides a clean, user-friendly web interface powered by Streamlit. When a user uploads an image of a flower, the system generates the following outputs sequentially:

1. **Original Image Display:** The uploaded image is displayed on the screen for visual confirmation.
2. **Species Prediction:** The CNN outputs the predicted flower species (e.g., "Daisy") alongside a progress bar indicating the model's confidence level (e.g., 96.5%).
3. **Color Analysis:** The app displays the extracted dominant RGB array (e.g., [245, 190, 10]).
    - It prints the mathematically closest human-readable color name based on the custom dictionary (e.g., "Blue").
    - A visual color swatch (a small colored square) is rendered on the screen to show the exact extracted color.

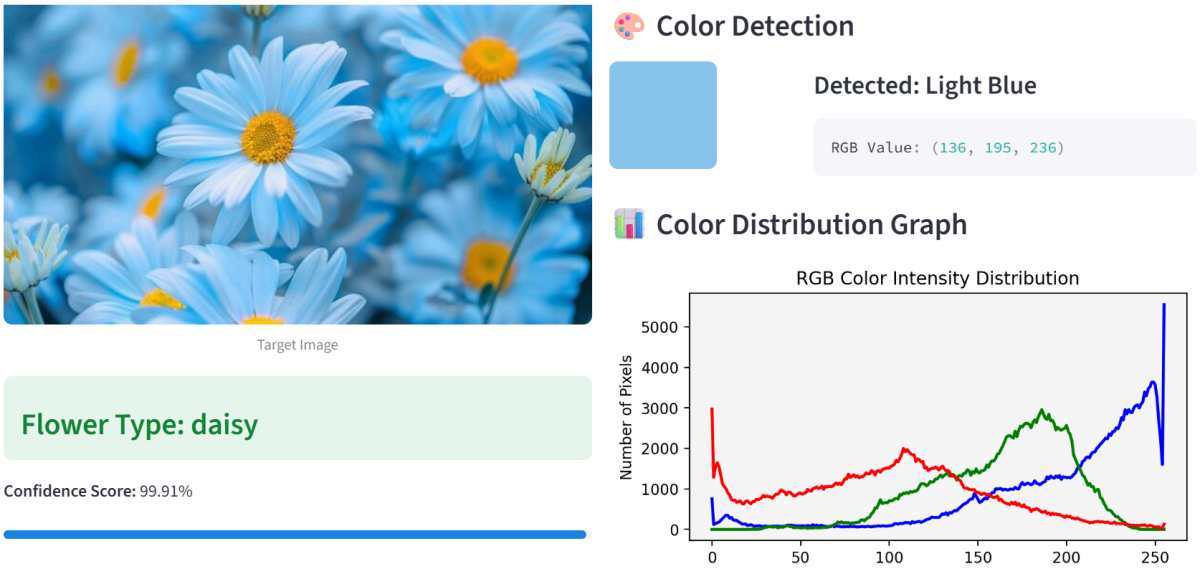

Figure 3: The Streamlit web application interface displaying the complete dual-pipeline analysis. The output showcases the original target image, the predicted flower species (Daisy) with its corresponding confidence score, the mathematically extracted dominant color profile, and an RGB color intensity distribution graph.

Conclusion and Future Work

Conclusion This project successfully demonstrates that complex image analysis can be performed efficiently in a lightweight web application. By utilizing MobileNetV2, the system achieves high accuracy in species classification without requiring immense computational power. Furthermore, the integration of background removal with K-Means clustering solves the common issue of background noise ruining color detection, resulting in highly accurate phenotypic data extraction.

Future Work There are several avenues for expanding this project in the future:

- **Dataset Expansion:** Training the CNN on a larger dataset to recognize hundreds of global flower species rather than a limited local set.
- **Multi-Color Palettes:** Upgrading the K-Means algorithm to n_clusters=3 or 5 to extract a complete color palette of the flower, identifying secondary colors like gradients or spots on the petals.
- **Cloud Deployment:** Deploying the Streamlit application to a cloud hosting platform (like AWS, Heroku, or Streamlit Community Cloud) so it can be accessed by users worldwide on any mobile device.